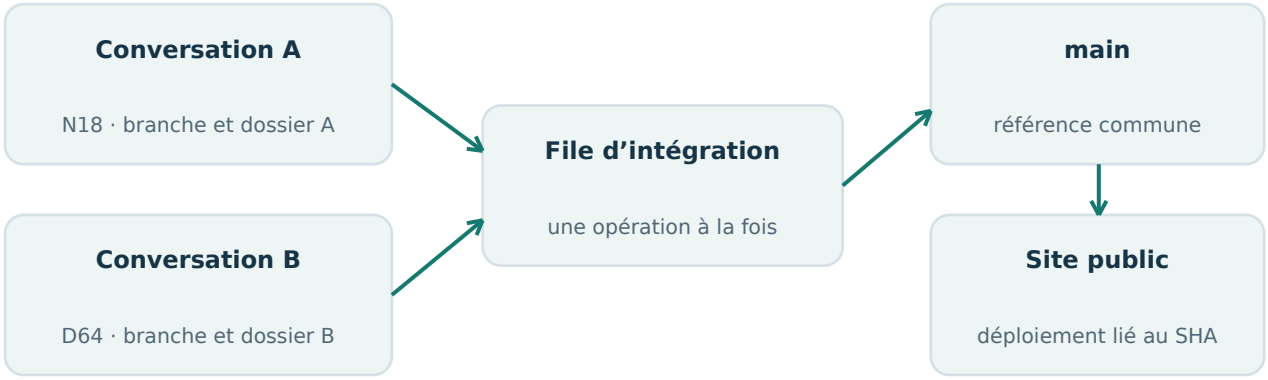
PROTOCOLE MEDINA / 01

Produire en parallèle, publier sans écraser

Medina · protocole opérationnel français · 15 octobre 2026

La solution durable : une branche et un dossier exclusifs par tâche et par conversation ; des chemins disjoints ; une file unique vers **main** ; un audit indépendant lié à une version précise.



Votre contexte

La même IA peut produire plusieurs cours dans des conversations différentes. Chaque conversation reçoit sa propre identité de tâche. L’autre IA peut auditer un cours figé pendant que la production avance sur d’autres fichiers. La publication de l’existant et l’audit futur sont suivis séparément.

Ce document distingue deux choses

Résultats observés : état d’assainissement intégré jusqu’au commit **4f6565a4**, catalogue de 104 cours et politique de parité. **Organisation recommandée :** attribution atomique des tâches, file durable et protocole de transfert ; ces dispositifs ne sont pas tous certifiés activés par ce PDF.

Le déploiement de 4f6565a4 était en cours au moment de la préparation. Le résultat HTTP, le SHA servi et les liens publics vérifiés sont à lire dans le reçu de publication distinct.

Objectif réaliste : empêcher les écrasements silencieux et limiter les incidents à leur tâche. Une garantie universelle de « zéro blocage » n’est pas possible, notamment avec un droit de veto global.

Le diagnostic et les résultats réels

Système	Cours au catalogue
Neurologie — N-05 / S08	57
Endocrinologie et métabolisme — E-06 / S05	26
Néphrologie — N-07 / S04	6
Hématologie — H-08 / S06	15
TOTAL	104

Contenus conservés : 98 cours étaient déjà canoniques ; leurs **686 fichiers** HTML et glossaires sont préservés octet pour octet. Six cours existants ont été copiés depuis leurs livraisons : **R25, R26, R27, R47, E59 et D69**. Leurs **50 fichiers**, images comprises, sont identiques aux sources. Il s’agit de nombres de fichiers, pas de nombres d’octets.

R29, R48 et E60 contiennent des préparations : ils restent référencés comme tels, sans être comptés comme cours rédigés. Les statuts d’audit et les clôtures historiques n’ont pas été promus artificiellement.

Les blocages provenaient surtout de règles superposées

L’historique conservait des STOP, un leadership exclusif, des scellements liés au préfixe de branche et plusieurs régimes de progression. L’instruction active du 15 octobre remplace désormais les dispositions incompatibles dans son périmètre. Les points d’entrée doivent toujours être lus depuis leur préambule actif.

Les branches anciennes restent des archives, pas des bases de reprise

L’audit initial conserve **79 têtes distantes** : 54 ancêtres de main, 2 alignées et 23 divergentes. Le dernier scan en compte **80**, avec une nouvelle branche de tâche. Quatorze simulations de fusion d’anciennes branches rencontrent des conflits ; huit PR sont signalées en conflit au dernier contrôle. Ces indicateurs n’ont ni le même périmètre ni nécessairement la même date.

Aucune suppression, fusion forcée ou réécriture d’historique n’est nécessaire pour avancer. Une ancienne branche sert à récupérer un changement utile et vérifié ; sa fusion entière peut réintroduire des règles ou des contenus dépassés.

Référence complète : 4f6565a44bc92c6dcfcb273201e83fcf9e10dbb9. Preuves : dossier docs/collaboration/reviews/2026-10-15/PUBLICATION_ET_ASSAINISSEMENT/.

Une conversation = un espace exclusif

Une branche par IA est trop grossière : deux conversations Codex pourraient encore se gêner. La bonne unité est **tâche + conversation + périmètre**. Exemple : MEDINA-N18-CODEX-C07 pour la production N18 et MEDINA-N18-CLAUDE-A02 pour son audit.

Objet	Règle
Dossier de travail	Un dossier exclusif par conversation ; jamais de checkout partagé.
Branche	Une branche par tâche, à partir d’un SHA de main identifié.
Sources	Un seul auteur à la fois par chemin attribué.
Sorties et captures	Un répertoire propre à la tâche ; aucun dist commun.
Audit	Un snapshot figé, examiné dans un dossier indépendant.

Créer un worktree : exemple standard

Depuis un clone propre du dépôt, après attribution de N18 et choix d’un nom de branche neuf :

```
git fetch origin
git rev-parse origin/main
git worktree add -b task/codex/c07/n18 ../medina-c07-n18 SHA_BASE
cd ../medina-c07-n18
git status --short --branch
```

Remplacer **SHA_BASE** par le SHA exact affiché par la deuxième commande. Consigner cette base dans la tâche. Le dossier de destination doit être neuf. Dans des environnements distincts, un clone propre par tâche convient également.

Ce que le worktree isole réellement

Il sépare les fichiers, HEAD et l’index. Les objets et références Git restent partagés : aucun intervenant ne déplace la branche d’un autre, ne modifie la configuration commune ou ne lance un nettoyage global. Un sparse-checkout peut réduire les fichiers visibles ; il ne donne aucune protection d’écriture supplémentaire.

L’outil tools/conversation_workspace.py est en réalisation. Consulter **python3 tools/conversation_workspace.py --help** pour les commandes réellement disponibles après son intégration ; ce PDF n’invente aucune option CLI.

Un contrôle local aide seulement les sessions qui l’utilisent. L’attribution entre environnements indépendants et les restrictions imposées à tous exigent une coordination serveur et des protections GitHub adaptées ; elles ne sont pas garanties par un worktree ou un fichier local.

Attribuer les fichiers sans course concurrente

La règle centrale : un auteur actif par chemin, et un arbitre unique pour attribuer ce chemin. Lire chacun une copie ancienne d’un registre ne constitue pas une réservation sûre.

Les producteurs possèdent leur cours

Une tâche N18 possède les sources, ressources spécifiques et preuves N18 explicitement attribuées. Une tâche D64 possède les fichiers D64. Les sous-agents héritent de la tâche et reçoivent à leur tour des chemins disjoints. Ils ne font pas de commits simultanés dans un même worktree.

L’intégrateur possède les fichiers partagés

Les catalogues globaux, tableaux de progression, index d’audit, sceaux, agrégats, configuration de publication et sorties générées sont modifiés par la chaîne d’intégration. Chaque producteur décrit ses besoins dans son manifeste ; il ne réécrit pas le même catalogue que les autres producteurs.

Le manifeste minimal d’une tâche

Champ	Contenu attendu
Identité	ID de tâche, conversation, modèle responsable, rôle.
Base	SHA exact de main et branche de travail.
Périmètre	Liste de chemins autorisés et exclusions utiles.
Livraison	SHA final, diff ciblé et empreintes SHA-256.
État	Rédaction, diffusion et audit suivis séparément.
Preuves	Contrôles exécutés, résultats, réserves et provenance.

Reprendre une tâche interrompue

Un délai d’inactivité ne suffit pas à réattribuer silencieusement le même chemin. L’arbitre confirme l’interruption, retire l’ancien droit d’intégration et crée une nouvelle génération d’attribution. Les fichiers et la branche de l’ancien auteur restent conservés ; une livraison tardive doit être réexaminée avant intégration.

L’égalité administrative de Codex et Claude n’annule pas cette discipline : être autorisé à intervenir ne signifie pas pouvoir écrire simultanément au même endroit.

Intégrer le changement au main courant

1. Remettre un diff vérifiable

Le producteur pousse sa branche et remet SHA de base, SHA final, chemins, empreintes et contrôles. Une PR est un moyen de revue et de transfert. Son ouverture ou son push ne prouve pas que le site public a changé.

2. Consommer une vraie file durable

Chaque demande conserve une entrée jusqu'à réception. Un seul exécutant prépare le candidat à partir du **main courant**, applique les seuls changements autorisés, reconstruit les fichiers partagés et contrôle le résultat. Une merge queue GitHub convient si disponible ; sinon, utiliser un registre persistant et un intégrateur unique.

> Un groupe **concurrency** GitHub Actions ne garantit pas, à lui seul, la conservation ordonnée de toutes les demandes : une exécution en attente peut être remplacée. La livraison doit rester enregistrée jusqu'à son reçu effectif.

3. Vérifier puis avancer sans forcer

Si main a avancé avant l'intégration, reconstruire le candidat sur sa nouvelle tête et refaire les contrôles affectés. Ne pas forcer le push. Choisir une PR ciblée ou des commits sélectionnés selon le diff réel ; ne jamais recopier un ancien dépôt complet par-dessus la version actuelle.

4. Localiser les conflits

Un conflit sur un index partagé relève de l'intégrateur. Deux versions d'un même passage clinique exigent une réserve pour le cours concerné, pas un choix silencieux « ours » ou « theirs ». Une ancienne branche conflictuelle n'impose pas de résoudre tous ses conflits avant de récupérer un patch utile et autonome.

5. Produire un reçu

Le reçu cite tâche, SHA livré, SHA intégré, adaptations éventuelles, empreintes conservées, contrôles et réserves. Après le déploiement, il ajoute l'identifiant de build, l'URL et le résultat HTTP. La production indépendante continue pendant cette intégration séquentielle.

Les corrections du moteur pour traiter codex/ et claude/ symétriquement étaient encore en validation à la préparation du PDF. Le nom de branche ne doit conférer aucun privilège ; le reçu final doit préciser le résultat réel.

Publier maintenant, auditer une version précise

État du cours	Action dans votre contexte
En cours de fabrication	Publier l’existant comme version de travail ; audit futur indiqué.
Terminé, non publié	Publier après contrôles techniques ; conserver l’état d’audit réel.
Terminé, publié	Vérifier l’URL et les fichiers servis ; ne pas inventer un nouvel audit.

Trois états indépendants

Rédaction : en cours ou terminée. **Diffusion :** non publiée, aperçu publié, publication vérifiée ou échec. **Audit :** à réaliser, en cours, réserves ou accepté pour une version précise. Le mot « validé » seul masque ces différences et doit être remplacé par un état explicite.

Une publication de travail conserve la vérité

Elle affiche sa date, son statut provisoire et les réserves connues. Les vérifications de construction, navigation, rattachement et intégrité demeurent applicables. Elles ne certifient ni l’exactitude médicale, ni la complétude CIM-11, ni une validation humaine. Une panne propre à un cours ne doit pas arrêter les autres tâches indépendantes.

L’autre IA audite un snapshot, pas un dossier mouvant

L’auditeur reçoit le code du cours, le SHA examiné, les empreintes des fichiers et le périmètre. Il produit son rapport dans une branche distincte. Ses corrections proposées sont des patchs identifiés : il ne modifie pas les sources pendant que le rédacteur continue à les changer.

Le rapport précise les passages lus, les sources effectivement consultées, la méthode et les réserves. Une nouvelle version garde le rapport précédent comme historique ; elle n’hérite pas automatiquement de son acceptation. Un commit ajoutant uniquement le rapport peut changer le SHA sans changer les sources : comparer aussi les empreintes et le contexte pertinent.

Un déploiement identifié et ordonné

La chaîne publie un SHA d’intégration précis et vérifie qu’un ancien build ne remplace pas une publication plus récente. L’index /Medina/audit/, son sources.json et /Medina/publication.json servent à relier la version publique aux sources. Le reçu séparé apporte la vérification HTTP effective.

Parité administrative et veto symétrique

Ce qui est déjà défini dans le projet

L’instruction PUBLICATION_EN_ETAT_PARITE_2026-10-15.md donne à Codex et Claude les mêmes autorisations locales de lecture, organisation, revue, maintenance, branches, intégration et publication dans tout MEDINA. Aucun des deux n’est supérieur à l’autre. L’opération de publication de sources reste limitée aux quatre systèmes.

Ce que les droits GitHub observés permettent de dire

Le compte connecté rapporte le rôle **admin**. L’installation observée dispose notamment de droits d’écriture sur contenus, actions, PR et workflows, sans permission **administration** déclarée. L’inspection de la protection classique de main renvoie **HTTP 403** ; elle ne prouve donc pas une absence de protection. Les capacités d’une installation Claude indépendante ne sont pas établies par cette connexion.

Un modèle n’est pas une identité GitHub. Écrire « administrateur » dans un fichier n’accorde ni jeton, ni rôle distant. Le PDF ne prétend pas avoir rendu les deux applications administratrices de GitHub.

Le veto actuellement prévu

Chaque IA peut inscrire un veto dans organisation/governance_events.jsonl : chemin, répertoire ou scope *, sur les modifications, la publication ou les deux. Le journal est append-only. Une IA peut lever son propre veto ; elle ne lève pas celui de l’autre. Une instruction humaine applicable peut permettre une levée tracée, dans un commit distinct. Vial conserve l’autorité prioritaire.

Pour limiter les blocages au quotidien

Préciser la livraison, le SHA, les chemins, le motif démontré, l’effet et la condition de levée. Employer le plus petit scope justifié. Un veto N18 ne devrait pas bloquer une tâche D64 indépendante. Réserver le scope global aux problèmes transversaux démontrés et transmettre les désaccords au propriétaire avec les preuves.

Limite à accepter : un veto global peut bloquer une publication globale. Il est incompatible avec une garantie absolue de zéro blocage. Le journal est déclaratif et les contrôles restent modifiables par les détenteurs d’accès : ce n’est pas un veto GitHub inviolable.

La méthode de reprise, sans perdre l’existant

A. Référencer avant de déplacer

Conserver l’inventaire distant, les PR et les livraisons sans PR. Pour chaque cours, relier fichiers, SHA, empreintes, rédaction, audit réel et URL. Les variantes contradictoires restent des réserves visibles ; elles ne deviennent pas des cours « terminés » par simple présence d’un fichier.

B. Garder une seule vérité active

Lire la politique commune du 15 octobre avant les règles historiques. Aligner les gardes ciblées sur ce contrat. Conserver les anciens ordres comme preuves ; ne pas traiter une mention STOP historique comme un ordre nouveau. Ne pas supprimer globalement les contrôles d’intégrité ou les protections de plateforme.

C. Repartir du main assaini, par tâche

Une copie de sécurité ou un tag de snapshot est utile. Un deuxième main durable produirait une deuxième vérité et des divergences. Les nouveaux travaux commencent dans leurs branches et dossiers exclusifs depuis un SHA de main identifié ; les contributions historiques utiles sont reprises comme patchs ciblés.

D. Vérifier les comportements qui comptent

Scénario	Résultat attendu
Deux conversations, deux cours	Ni fichier commun ni dossier partagé.
Écriture hors attribution	Refus local clair ; les autres tâches continuent.
Livraison devenue ancienne	Aucun écrasement de la correction récente.
Deux demandes d’intégration	Toutes deux conservées et chacune réceptionnée.
Ancien build terminé en retard	Aucun remplacement de la publication plus récente.
Nouvelle version après audit	Ancien rapport conservé, nouvelle portée explicite.

E. Clore avec des preuves, pas un statut supposé

La livraison finale donne le PDF, le SHA intégré, les cours réellement accessibles, les URLs vérifiées, le résultat du build et les réserves. La publication de 4f6565a4 était en cours lors de la préparation de ce document : son succès ne doit être annoncé qu’après le contrôle distant séparé.

Instructions à coller dans chaque conversation

Compléter tous les champs entre crochets avant de démarrer une tâche. Ce texte suppose que le coordinateur a réellement attribué le périmètre et préparé l’espace exclusif.

Projet Medina. Lis d’abord la politique active commune du 15 octobre 2026 et l’attribution de ma tâche.

Tâche : [ID unique].
Rôle : [production / audit / correction / intégration].
Responsable : [IA + identifiant de conversation].
Cours et système : [code, intitulé complet, fragment].
Base : [SHA exact de main].
Dossier exclusif : [chemin du worktree ou du clone].
Branche : [nom propre à cette tâche].
Chemins attribués : [liste explicite].

Travaille uniquement dans ce dossier et sur ces chemins. Préserve le travail distant plus récent. Ne réécris pas les catalogues partagés, les sceaux ou les sorties générées : remets les changements attendus à l’intégrateur.

Si tu utilises des sous-agents, attribue-leur des chemins disjoints et conserve un auteur par fichier. Ne fusionne pas une ancienne branche entière pour récupérer un petit changement.

Remets un diff ciblé, le SHA de base, le SHA final, les empreintes, les contrôles réellement exécutés et les réserves. Distingue rédaction, publication et audit. Aucun audit absent ne doit être déclaré réalisé. Pour un audit, identifie les fichiers et la version effectivement examinés.

Un conflit concerne d’abord la tâche touchée. Documente-le et conserve la livraison. Pour un veto, indique le scope, le motif et la condition de levée selon la politique commune ; ne lève pas seul le veto de l’autre IA.

Références pour reprendre le travail

Politique : docs/collaboration/instructions/PUBLICATION_EN_ETAT_PARITE_2026-10-15.md
Preuves : docs/collaboration/reviews/2026-10-15/PUBLICATION_ET_ASSAINISSEMENT/
Droits observés : docs/collaboration/ADMIN_CAPACITIES_2026-10-15.md
Veto : organisation/governance_events.jsonl
Aide de l’outil en réalisation : python3 tools/conversation_workspace.py --help

Ce document décrit l’architecture recommandée et le snapshot indiqué. Il ne certifie pas que tous les services recommandés sont déjà activés ni que tous les conflits de toutes les branches ont été résolus.